

CS 634
Data Mining
Midterm Project--Brute Force/Apriori Algorithms
for Association Rule Mining

Zoe Sedlacek

Table of Contents

Project Description --	3
Summary --	3
Source Code & Annotations --	5
Input Reading & Setup --	5
Apriori & Brute Force --	7
Result Formatting --	11
Datasets & Results --	15

Project Description

The goal of this project was to implement the Brute Force algorithm for finding association rules and compare the results to an existing Apriori implementation, confirming that the resulting rules are the same and that Brute Force is less time efficient than Apriori. The submission includes the python code, in both python script and Jupyter Notebook format, five datasets, each containing 30 items and 20 transactions, in csv format, and this report document. The results include the frequent itemsets and association rules for each dataset generated by both algorithms, as well as the time taken by both algorithms to run.

Summary

The coding language for this project is Python, written and tested in version 3.12. Dependencies include mlxtend, pandas, itertools, and time. Ensure that all requirements are installed and up to date before running the program.

The Brute Force and Apriori algorithms determine the frequent itemsets and association rules given a dataset of transactions, a minimum support, and a minimum confidence. A frequent itemset is defined as a set of items whose support(number of transactions including itemset / total transaction count) is greater than or equal to the minimum support threshold. An association rule is a relationship between sets of items $A \rightarrow B$ where the confidence(support of AB / support of A) is greater than or equal to the minimum confidence threshold. The Apriori algorithm does not check itemsets for frequency if any of their subsets are not frequent, while Brute Force checks all possible itemsets, only breaking when no itemsets of a length are frequent. This means that Brute Force should be checking many more itemsets for frequency than Apriori and should take more time.

The application takes in a data file name, minimum support level, and minimum confidence level from the user during runtime. The data file name must be a path to a valid, readable file in csv format and the minimum thresholds should be float values between 0 and 1. I recommend keeping the data files in the same folder as the application so you can simply input the data file name when queried. Each line of the data file is considered a separate transaction with items separated by commas. If you are using your own data, ensure it is in this format or you may get erroneous results. Some example transactions are pictured below. The input to both algorithms includes the transaction information, encoded into one-hot format, and the minimum support and confidence levels.

```
Tofu,Almonds,Honey,Pasta,Ground Beef,Spinach  
Eggs,Chicken Breast,Cheese,Yogurt,Spinach,Rice  
Apples,Orange Juice,Honey,Potatoes,Chips,Cereal  
Bananas,Spinach,Chocolate Bar,Ice Cream  
Bread,Pasta,Yogurt,Tomato Sauce,Chips,Frozen Vegetables
```

Source Code & Annotations

Input Reading & Setup

Dependencies include pandas for dataframes, mlxtend for encoding and apriori algorithm, itertools for combination finding, and time for measuring the length of algorithm runtime. They must all be installed and up to date to run the program.

```
#pip install pandas #use if need to install
#pip install mlxtend
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
from itertools import combinations
from time import process_time
```

The file name, minimum support, and minimum confidence are taken as input from the user. The file given must be a valid path and readable to continue.

```
fileName = input('Enter data file name ')
fileCheck = 0
while (not fileCheck):
    try:
        file = open(fileName, "r")
    except:
        fileName = input('File could not be opened. Enter new file name ')
    else:
        fileCheck = 1
```

```
Enter data file name  projdata1.txt
```

```
minSup = float(input('Enter min support value '))
```

```
Enter min support value  0.2
```

```
minCon = float(input('Enter min confidence value '))
```

```
Enter min confidence value  0.5
```

The file data is read into an array of arrays. Each line is considered one transaction with items split by commas. All transactions are printed to the screen after being read in.

```

file = open(fileName)
#file = open('projdata1.txt') #For faster testing, comment out
transactions = []
line = file.readline()
#Remove trailing space, split into attributes, add to list
while(line != ''):
    line = line.strip()
    splitLine = line.split(',')
    transactions.append(splitLine)
    line = file.readline()
print("Transactions:")
for t in transactions:
    print(t)
print('\n')

```

```

Transactions:
['Apples', 'Bananas', 'Milk', 'Eggs', 'Chicken Breast', 'Cheese']
['Bread', 'Eggs', 'Cheese', 'Yogurt', 'Orange Juice', 'Cereal']
['Rice', 'Tomato Sauce', 'Yogurt', 'Cereal', 'Frozen Vegetables', 'Chips']
['Carrots', 'Cereal', 'Ice Cream', 'Chocolate Bar', 'Pasta']
['Chicken Breast', 'Peanut Butter', 'Lettuce', 'Honey']
['Bananas', 'Bread', 'Tomato Sauce', 'Onions', 'Pasta', 'Fish']
['Apples', 'Carrots', 'Pasta', 'Frozen Vegetables', 'Cheese', 'Rice']
['Milk', 'Cheese', 'Orange Juice', 'Yogurt', 'Chocolate Bar']
['Chocolate Bar', 'Spinach', 'Onions', 'Potatoes', 'Eggs', 'Fish']
['Fish', 'Rice', 'Ground Beef', 'Yogurt', 'Frozen Vegetables', 'Peanut Butter']
['Apples', 'Yogurt', 'Almonds', 'Ice Cream', 'Honey']
['Bananas', 'Tomato Sauce', 'Cereal', 'Frozen Vegetables', 'Milk', 'Cheese']
['Bread', 'Chicken Breast', 'Fish', 'Garlic', 'Chips', 'Peanut Butter']
['Carrots', 'Rice', 'Cereal', 'Ice Cream', 'Orange Juice', 'Salsa']
['Pasta', 'Frozen Vegetables', 'Cheese', 'Milk', 'Eggs', 'Chocolate Bar']
['Tofu', 'Almonds', 'Honey', 'Pasta', 'Ground Beef', 'Spinach']
['Eggs', 'Chicken Breast', 'Cheese', 'Yogurt', 'Spinach', 'Rice']
['Apples', 'Orange Juice', 'Honey', 'Potatoes', 'Chips', 'Cereal']
['Bananas', 'Spinach', 'Chocolate Bar', 'Ice Cream']
['Bread', 'Pasta', 'Yogurt', 'Tomato Sauce', 'Chips', 'Frozen Vegetables']

```

Both algorithm implementations require an input format of a dataframe with one-hot encoded information about the transactions. The mlxtend transaction encoder is used for this transformation. This one-hot encoded transaction dataframe is shown below, with items in the dataset as columns and True and False used to mark if an item is/is not in a transaction.

```
#use transaction encoder to get one hot dataframe of attributes
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
transactionDF = pd.DataFrame(te_ary, columns=te.columns_)
```

transactionDF

	Almonds	Apples	Bananas	Bread	Carrots	Cereal	Cheese	Chicken Breast	Chips	Chocolate Bar	Eggs	Fish	Frozen Vegetables	Garlic	Ground Beef	Honey
0	False	True	True	False	False	False	True	True	False	False	True	False	False	False	False	False
1	False	False	False	True	False	True	True	False	False	False	True	False	False	False	False	False
2	False	False	False	False	False	True	False	False	True	False	False	False	True	False	False	False
3	False	False	False	False	True	True	False	False	False	True	False	False	False	False	False	False
4	False	False	False	False	False	False	False	True	False	False	False	False	False	False	False	True
5	False	False	True	True	False	False	False	False	False	False	False	True	False	False	False	False
6	False	True	False	False	True	False	True	False	False	False	False	False	True	False	False	False
7	False	False	False	False	False	False	True	False	False	True	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False	False	True	True	True	False	False	False	False
9	False	False	False	False	False	False	False	False	False	False	False	True	True	False	True	False
10	True	True	False	False	False	False	False	False	False	False	False	False	False	False	False	True
11	False	False	True	False	False	True	True	False	False	False	False	False	True	False	False	False
12	False	False	False	True	False	False	False	True	True	False	False	True	False	True	False	False
13	False	False	False	False	True	True	False	False	False	False	False	False	False	False	False	False
14	False	False	False	False	False	False	True	False	False	True	True	False	True	False	False	False
15	True	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True
16	False	False	False	False	False	False	True	True	False	False	True	False	False	False	False	False
17	False	True	False	False	False	True	False	False	True	False	False	False	False	False	False	True
18	False	False	True	False	False	False	False	False	False	True	False	False	False	False	False	False
19	False	False	False	True	False	False	False	False	True	False	False	False	True	False	False	False

Apriori & Brute Force

For the apriori algorithm, the start and end times for the algorithm are stored to calculate the time taken and the frequent itemsets and association rules are generated from the prebuilt implementation. The time taken includes only the time to generate the frequent itemsets, as the method for finding frequent itemsets is the only variation between the algorithms and is sufficient for demonstrating that Apriori is more time efficient than Brute Force.

I added a 'length' column to the Apriori output, with the value being the amount of items in each frequent itemset, for easier comparison with the Brute Force output.

```
#Apriori -- use mlxtend implementation to check freq itemsets and association rules
apStart = process_time()
freqItemsets = apriori(transactionDF, min_support=minSup, use_colnames=True)
freqItemsets['length'] = freqItemsets['itemsets'].apply(lambda x: len(x))
apEnd = process_time()

mlxRuleDf = association_rules(freqItemsets, metric="confidence", min_threshold=minCon)
```

My Brute Force implementation loops through the number of items possible in a frequent itemset(1 to number of items in the dataset).

```
#brute force
bfStart = process_time()
#combinations & support
#len(attList) + 1
totalTransactions = len(transactionDF)
attList = list(transactionDF.columns)
supDF = pd.DataFrame()
#For each number of attributes
for numAtt in range(1, len(attList) + 1):
    freqItemBool = False
    #For each combination of number of attributes
```

Each possible combination of items of length K is checked to see if it is frequent by finding the number of transactions which include all the items of the combination, dividing by the number of total transactions, and seeing if that fraction is greater than or equal to the minimum support. Frequent itemsets are added to the result dataframe along with their support. For example, if checking for frequent itemsets with items A, B, and C and K = 2, the possibilities would be AB, BC, and AC.


```

for combo in list(combinations(attList, numAtt)):
    prev = 1
    #Create df for combination by and-ing df of all attributes in it
    for att in combo:
        attDF = transactionDF[att] & prev
        prev = attDF
    #display(combo)
    #Remove rows that do not have all attributes in combo
    comboDF = transactionDF[attDF]
    support = len(comboDF) / len(transactionDF)
    if support >= minSup:
        freqItemBool = True
        row = {'combo': combo, 'support': support, 'length': numAtt}
        supDF = pd.concat([supDF, pd.DataFrame([row])], ignore_index=True)

```

If there are no frequent itemsets of a length K , the algorithm terminates without checking possible itemsets of length $K + 1$.

```

#if, after all combos of length X have been checked, none are frequent, brute force breaks
if freqItemBool == False:
    break
bfEnd = process_time()

```

```

#brute force
bfStart = process_time()
#combinations & support
#len(attList) + 1
totalTransactions = len(transactionDF)
attList = list(transactionDF.columns)
supDF = pd.DataFrame()
#For each number of attributes
for numAtt in range(1, len(attList) + 1):
    freqItemBool = False
    #For each combination of number of attributes
    for combo in list(combinations(attList, numAtt)):
        prev = 1
        #Create df for combination by and-ing df of all attributes in it
        for att in combo:
            attDF = transactionDF[att] & prev
            prev = attDF
        #display(combo)
        #Remove rows that do not have all attributes in combo
        comboDF = transactionDF[attDF]
        support = len(comboDF) / len(transactionDF)
        if support >= minSup:
            freqItemBool = True
            row = {'combo': combo, 'support': support, 'length': numAtt}
            supDF = pd.concat([supDF, pd.DataFrame([row])], ignore_index=True)
    #if, after all combos of length X have been checked, none are frequent, brute force breaks
    if freqItemBool == False:
        break
bfEnd = process_time()

```

Association rules are found for Brute Force by checking all frequent itemsets of length 2 or greater for possible rules.

```
#Check rules
myRuleDF = pd.DataFrame()
for length in range(2, max(supDF['length']) + 1):
    for freqCombo in supDF[supDF['length'] == length]['combo']:
        #print(freqCombo)
        freqComboSup = supDF.loc[supDF['combo'] == freqCombo]['support'].iloc[0]
        #print("Support = " + str(freqComboSup))
        #print("Frequent Subsets:")
        #print('\n')
```

Each possible antecedent(left hand side) is checked for all its possible consequents(right hand side). Right hand sides are only considered if they do not include items from the left hand side (ex. A -> AB is not valid, A -> B is valid)

```
for subsetLength in range(1, length):
    for subsetCombo in list(combinations(freqCombo, subsetLength)):
        #print(subsetCombo)
        subsetSup = supDF.loc[supDF['combo'] == subsetCombo]['support'].iloc[0]
        #print("Support = " + str(subsetSup))

        #for each freq subset on the lhs, try all freq subsets as possible rhs
        #print("possible rhs:")
        for rhsLength in range(1, length + 1):
            for rhs in list(combinations(freqCombo, rhsLength)):
                rhsSup = supDF.loc[supDF['combo'] == rhs]['support'].iloc[0]
                validRule = True
                #Remove rhs from consideration if it includes the lhs
                for subsetAttribute in subsetCombo:
                    if subsetAttribute in rhs:
                        validRule = False
```

For each possible rule, the confidence is calculated by dividing the support of the left and right side together by the support of just the left hand side. If the confidence is greater than or equal to the minimum confidence, the rule is added to the result rule dataframe along with its supports and confidence.

```

        if validRule:
            #print(rhs)
            totalTup = tuple(sorted(subsetCombo + rhs))
            totalSup = supDF.loc[supDF['combo'] == totalTup]['support'].iloc[0]
            if totalSup / subsetSup >= minCon:
                ruleRow = {'antecedents': subsetCombo, 'consequents': rhs, 'antecedent support': subsetSup, 'consequent support': rhsSup,
                           'support': totalSup, 'confidence': totalSup / subsetSup}
                myRuleDF = pd.concat([myRuleDF, pd.DataFrame([ruleRow])], ignore_index=True)
            #print("RULE")
            #print(str(subsetCombo) + "-->" + str(rhs))
            #print("Confidence = " + str(freqComboSup / subsetSup))

#Check rules
myRuleDF = pd.DataFrame()
for length in range(2, max(supDF['length'] + 1):
    for freqCombo in supDF[supDF['length'] == length]['combo']:
        #print(freqCombo)
        freqComboSup = supDF.loc[supDF['combo'] == freqCombo]['support'].iloc[0]
        #print("Support = " + str(freqComboSup))
        #print("Frequent Subsets:")
        #print('\n')
        for subsetLength in range(1, length):
            for subsetCombo in list(combinations(freqCombo, subsetLength)):
                #print(subsetCombo)
                subsetSup = supDF.loc[supDF['combo'] == subsetCombo]['support'].iloc[0]
                #print("Support = " + str(subsetSup))

                #for each freq subset on the lhs, try all freq subsets as possible rhs
                #print("possible rhs:")
                for rhsLength in range(1, length + 1):
                    for rhs in list(combinations(freqCombo, rhsLength)):
                        rhsSup = supDF.loc[supDF['combo'] == rhs]['support'].iloc[0]
                        validRule = True
                        #Remove rhs from consideration if it includes the lhs
                        for subsetAttribute in subsetCombo:
                            if subsetAttribute in rhs:
                                validRule = False
                        if validRule:
                            #print(rhs)
                            totalTup = tuple(sorted(subsetCombo + rhs))
                            totalSup = supDF.loc[supDF['combo'] == totalTup]['support'].iloc[0]
                            if totalSup / subsetSup >= minCon:
                                ruleRow = {'antecedents': subsetCombo, 'consequents': rhs, 'antecedent support': subsetSup, 'consequent support': rhsSup,
                                           'support': totalSup, 'confidence': totalSup / subsetSup}
                                myRuleDF = pd.concat([myRuleDF, pd.DataFrame([ruleRow])], ignore_index=True)
                            #print("RULE")
                            #print(str(subsetCombo) + "-->" + str(rhs))
                            #print("Confidence = " + str(freqComboSup / subsetSup))

                #print("\n")
            #print("-----")

```

Result Formatting

The resulting frequent itemsets and association rules for both algorithms are printed out to the console. This is done by simply printing the dataframes where the results were stored during the algorithm runtimes. The Brute Force rule dataframe is checked for duplicates before printing in case multiple frequent itemsets implied the same rules.

```
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
print("Apriori Frequent Itemsets")
print(freqItemsets)
print("\n")
print("Apriori Rules")
print(mlxRuleDf)
print("\n")
print("Brute Force Frequent Itemsets")
print(supDF)
print("\n")
print("Brute Force Rules")
print(myRuleDF.drop_duplicates())
print("\n")
```

Apriori Frequent Itemsets

	support	itemsets	length
0	0.20	(Apples)	1
1	0.20	(Bananas)	1
2	0.20	(Bread)	1
3	0.30	(Cereal)	1
4	0.35	(Cheese)	1
5	0.20	(Chicken Breast)	1
6	0.20	(Chips)	1
7	0.25	(Chocolate Bar)	1
8	0.25	(Eggs)	1
9	0.20	(Fish)	1
10	0.30	(Frozen Vegetables)	1
11	0.20	(Honey)	1
12	0.20	(Ice Cream)	1
13	0.20	(Milk)	1
14	0.20	(Orange Juice)	1
15	0.30	(Pasta)	1
16	0.25	(Rice)	1
17	0.20	(Spinach)	1
18	0.20	(Tomato Sauce)	1
19	0.35	(Yogurt)	1
20	0.20	(Cheese, Eggs)	2
21	0.20	(Milk, Cheese)	2

Apriori Rules

	antecedents	consequents	antecedent support	consequent support	support \
0	(Cheese)	(Eggs)	0.35	0.25	0.2
1	(Eggs)	(Cheese)	0.25	0.35	0.2
2	(Milk)	(Cheese)	0.20	0.35	0.2
3	(Cheese)	(Milk)	0.35	0.20	0.2

	confidence	lift	leverage	conviction	zhangs_metric
0	0.571429	2.285714	0.1125	1.750000	0.865385
1	0.800000	2.285714	0.1125	3.250000	0.750000
2	1.000000	2.857143	0.1300	inf	0.812500
3	0.571429	2.857143	0.1300	1.866667	1.000000

Brute Force Frequent Itemsets

	combo	support	length
0	(Apples,)	0.20	1
1	(Bananas,)	0.20	1
2	(Bread,)	0.20	1
3	(Cereal,)	0.30	1
4	(Cheese,)	0.35	1
5	(Chicken Breast,)	0.20	1
6	(Chips,)	0.20	1
7	(Chocolate Bar,)	0.25	1
8	(Eggs,)	0.25	1
9	(Fish,)	0.20	1
10	(Frozen Vegetables,)	0.30	1
11	(Honey,)	0.20	1
12	(Ice Cream,)	0.20	1
13	(Milk,)	0.20	1
14	(Orange Juice,)	0.20	1
15	(Pasta,)	0.30	1
16	(Rice,)	0.25	1
17	(Spinach,)	0.20	1
18	(Tomato Sauce,)	0.20	1
19	(Yogurt,)	0.35	1
20	(Cheese, Eggs)	0.20	2
21	(Cheese, Milk)	0.20	2

Brute Force Rules

	antecedents	consequents	antecedent support	consequent support	support \
0	(Cheese,)	(Eggs,)	0.35	0.25	0.2
1	(Eggs,)	(Cheese,)	0.25	0.35	0.2
2	(Cheese,)	(Milk,)	0.35	0.20	0.2
3	(Milk,)	(Cheese,)	0.20	0.35	0.2

confidence

0	0.571429
1	0.800000
2	0.571429
3	1.000000

Finally, the time taken for each algorithm is calculated by subtracting the start time from the end time. The percentage of time Apriori took compared to Brute Force is also displayed.

```
print("Apriori Time: " + str(apEnd - apStart))
print("Brute Force Time: " + str(bfEnd - bfStart))
print("Apriori took " + str(100 * (apEnd - apStart) / (bfEnd - bfStart)) + " percent as long as Brute Force")
```

```
Apriori Time: 0.03125
Brute Force Time: 1.359375
Apriori took 2.2988505747126435 percent as long as Brute Force
```

```
print('\n')
input("Press 'Enter' to exit")
```

Datasets & Results

This section includes all datasets, their frequent itemsets and association rules as given by both Brute Force and Apriori, and the time taken by both algorithms. The values for minimum support and minimum confidence used were 0.2 and 0.5 respectively. As expected, the frequent itemsets and rules generated by both algorithms were always the same. Brute force always took longer than Apriori, with the time difference growing larger as the length of frequent itemsets increased. The exact times may vary depending on your system. In my testing, running the script in Jupyter Notebook caused both algorithms to take slightly longer than running in the command line. The following results come from the command line.

projdata1

```
Transactions:
['Apples', 'Bananas', 'Milk', 'Eggs', 'Chicken Breast', 'Cheese']
['Bread', 'Eggs', 'Cheese', 'Yogurt', 'Orange Juice', 'Cereal']
['Rice', 'Tomato Sauce', 'Yogurt', 'Cereal', 'Frozen Vegetables', 'Chips']
['Carrots', 'Cereal', 'Ice Cream', 'Chocolate Bar', 'Pasta']
['Chicken Breast', 'Peanut Butter', 'Lettuce', 'Honey']
['Bananas', 'Bread', 'Tomato Sauce', 'Onions', 'Pasta', 'Fish']
['Apples', 'Carrots', 'Pasta', 'Frozen Vegetables', 'Cheese', 'Rice']
['Milk', 'Cheese', 'Orange Juice', 'Yogurt', 'Chocolate Bar']
['Chocolate Bar', 'Spinach', 'Onions', 'Potatoes', 'Eggs', 'Fish']
['Fish', 'Rice', 'Ground Beef', 'Yogurt', 'Frozen Vegetables', 'Peanut Butter']
['Apples', 'Yogurt', 'Almonds', 'Ice Cream', 'Honey']
['Bananas', 'Tomato Sauce', 'Cereal', 'Frozen Vegetables', 'Milk', 'Cheese']
['Bread', 'Chicken Breast', 'Fish', 'Garlic', 'Chips', 'Peanut Butter']
['Carrots', 'Rice', 'Cereal', 'Ice Cream', 'Orange Juice', 'Salsa']
['Pasta', 'Frozen Vegetables', 'Cheese', 'Milk', 'Eggs', 'Chocolate Bar']
['Tofu', 'Almonds', 'Honey', 'Pasta', 'Ground Beef', 'Spinach']
['Eggs', 'Chicken Breast', 'Cheese', 'Yogurt', 'Spinach', 'Rice']
['Apples', 'Orange Juice', 'Honey', 'Potatoes', 'Chips', 'Cereal']
['Bananas', 'Spinach', 'Chocolate Bar', 'Ice Cream']
['Bread', 'Pasta', 'Yogurt', 'Tomato Sauce', 'Chips', 'Frozen Vegetables']
```



```

Apriori Frequent Itemsets
  support      itemsets  length
0      0.20      (Apples)      1
1      0.20      (Bananas)     1
2      0.20      (Bread)       1
3      0.30      (Cereal)       1
4      0.35      (Cheese)        1
5      0.20      (Chicken Breast) 1
6      0.20      (Chips)         1
7      0.25      (Chocolate Bar) 1
8      0.25      (Eggs)          1
9      0.20      (Fish)           1
10     0.30      (Frozen Vegetables) 1
11     0.20      (Honey)          1
12     0.20      (Ice Cream)       1
13     0.20      (Milk)            1
14     0.20      (Orange Juice)    1
15     0.30      (Pasta)           1
16     0.25      (Rice)            1
17     0.20      (Spinach)         1
18     0.20      (Tomato Sauce)    1
19     0.35      (Yogurt)          1
20     0.20      (Eggs, Cheese)    2
21     0.20      (Milk, Cheese)    2

Apriori Rules
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0      (Eggs)      (Cheese)      0.25      0.35      0.2      0.800000      2.285714      0.1125      3.250000      0.750000
1      (Cheese)     (Eggs)      0.35      0.25      0.2      0.571429      2.285714      0.1125      1.750000      0.865385
2      (Milk)      (Cheese)    0.20      0.35      0.2      1.000000      2.857143      0.1300      inf          0.812500
3      (Cheese)     (Milk)      0.35      0.20      0.2      0.571429      2.857143      0.1300      1.866667      1.000000

Brute Force Frequent Itemsets
  combo      support  length
0      (Apples,)      0.20      1
1      (Bananas,)     0.20      1
2      (Bread,)        0.20      1
3      (Cereal,)       0.30      1
4      (Cheese,)        0.35      1
5      (Chicken Breast,) 0.20      1
6      (Chips,)         0.20      1
7      (Chocolate Bar,) 0.25      1
8      (Eggs,)          0.25      1
9      (Fish,)          0.20      1
10     (Frozen Vegetables,) 0.30      1
11     (Honey,)         0.20      1
12     (Ice Cream,)     0.20      1
13     (Milk,)          0.20      1
14     (Orange Juice,)  0.20      1
15     (Pasta,)         0.30      1
16     (Rice,)          0.25      1
17     (Spinach,)       0.20      1
18     (Tomato Sauce,)  0.20      1
19     (Yogurt,)        0.35      1
20     (Cheese, Eggs)   0.20      2
21     (Cheese, Milk)   0.20      2

Brute Force Rules
antecedents consequents antecedent support consequent support support confidence
0      (Cheese,)      (Eggs,)      0.35      0.25      0.2      0.571429
1      (Eggs,)      (Cheese,)    0.25      0.35      0.2      0.800000
2      (Cheese,)     (Milk,)      0.35      0.20      0.2      0.571429
3      (Milk,)      (Cheese,)    0.20      0.35      0.2      1.000000

Apriori Time: 0.0
Brute Force Time: 1.296875
Apriori took 0.0 percent as long as Brute Force

```

projdata2

```

Transactions:
['Laptop', 'Smartphone', 'Headphones', 'Tablet', 'Printer']
['Bluetooth Speaker', 'Desktop Computer', 'USB Flash Drive']
['Wireless Mouse', 'Monitor', 'Camera', 'Gaming Console']
['VR Headset', 'Home Router', 'Projector', 'Web Camera', 'Microphone']
['Charging Station', 'Laptop', 'Smartwatch', 'Graphics Card']
['Graphics Card', 'Motherboard', 'Power Bank']
['Phone Case', 'Screen Protector', 'HDMI Cable', 'Gaming Chair']
['Surge Protector', 'Laptop', 'Tablet', 'Headphones']
['Bluetooth Speaker', 'Desktop Computer', 'External Hard Drive']
['USB Flash Drive', 'Wireless Mouse', 'Monitor', 'Laptop']
['Keyboard', 'Camera', 'Gaming Console', 'VR Headset']
['Laptop', 'Smartphone', 'Headphones', 'Tablet', 'Charging Station']
['Desktop Computer', 'USB Flash Drive', 'Wired Mouse']
['Projector', 'Web Camera', 'Microphone', 'Power Bank']
['Graphics Card', 'Motherboard', 'Monitor']
['Phone Case', 'Screen Protector', 'Gaming Chair']
['Surge Protector', 'Laptop', 'Tablet', 'Headphones']
['Bluetooth Speaker', 'Desktop Computer', 'External Hard Drive']
['Wireless Mouse', 'Monitor', 'Camera', 'Gaming Console']
['VR Headset', 'Home Router', 'Projector', 'Web Camera']

```

```

Apriori Frequent Itemsets
support      itemsets      length
0      0.2      (Desktop Computer)      1
1      0.2      (Headphones)      1
2      0.3      (Laptop)      1
3      0.2      (Monitor)      1
4      0.2      (Tablet)      1
5      0.2      (Laptop, Headphones)      2
6      0.2      (Headphones, Tablet)      2
7      0.2      (Laptop, Tablet)      2
8      0.2      (Headphones, Laptop, Tablet)      3

```

```

Apriori Rules
antecedents      consequents      antecedent support      consequent support      support      confidence      lift      leverage      conviction      zhangs_metric
0      (Laptop)      (Headphones)      0.3      0.2      0.2      0.666667      3.333333      0.14      2.4      inf      1.000
1      (Headphones)      (Laptop)      0.2      0.3      0.2      1.000000      3.333333      0.14      inf      0.875      1.000
2      (Headphones)      (Tablet)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000
3      (Tablet)      (Headphones)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000
4      (Laptop)      (Tablet)      0.3      0.2      0.2      0.666667      3.333333      0.14      2.4      inf      1.000
5      (Tablet)      (Laptop)      0.2      0.3      0.2      1.000000      3.333333      0.14      inf      0.875      1.000
6      (Laptop, Headphones)      (Tablet)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000
7      (Tablet, Headphones)      (Laptop)      0.2      0.3      0.2      1.000000      3.333333      0.14      inf      0.875      1.000
8      (Tablet, Laptop)      (Headphones)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000
9      (Headphones)      (Tablet, Laptop)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000
10      (Laptop)      (Tablet, Headphones)      0.3      0.2      0.2      0.666667      3.333333      0.14      2.4      inf      1.000
11      (Tablet)      (Laptop, Headphones)      0.2      0.2      0.2      1.000000      5.000000      0.16      inf      1.000      1.000

```

```

Brute Force Frequent Itemsets
combo      support      length
0      (Desktop Computer,)      0.2      1
1      (Headphones,)      0.2      1
2      (Laptop,)      0.3      1
3      (Monitor,)      0.2      1
4      (Tablet,)      0.2      1
5      (Headphones, Laptop)      0.2      2
6      (Headphones, Tablet)      0.2      2
7      (Laptop, Tablet)      0.2      2
8      (Headphones, Laptop, Tablet)      0.2      3

```

```

Brute Force Rules
antecedents      consequents      antecedent support      consequent support      support      confidence
0      (Headphones,)      (Laptop,)      0.2      0.3      0.2      1.000000
1      (Laptop,)      (Headphones,)      0.3      0.2      0.2      0.666667
2      (Headphones,)      (Tablet,)      0.2      0.2      0.2      1.000000
3      (Tablet,)      (Headphones,)      0.2      0.2      0.2      1.000000
4      (Laptop,)      (Tablet,)      0.3      0.2      0.2      0.666667
5      (Tablet,)      (Laptop,)      0.2      0.3      0.2      1.000000
6      (Headphones,)      (Laptop, Tablet)      0.2      0.2      0.2      1.000000
7      (Laptop,)      (Headphones, Tablet)      0.3      0.2      0.2      0.666667
8      (Tablet,)      (Headphones, Laptop)      0.2      0.2      0.2      1.000000
9      (Headphones, Laptop)      (Tablet,)      0.2      0.2      0.2      1.000000
10      (Headphones, Tablet)      (Laptop,)      0.2      0.3      0.2      1.000000
11      (Laptop, Tablet)      (Headphones,)      0.2      0.2      0.2      1.000000

```

```

Apriori Time: 0.0
Brute Force Time: 10.921875
Apriori took 0.0 percent as long as Brute Force

```

projdata3

```

Transactions:
['Dog Food', 'Cat Food', 'Bird Seed', 'Fish Flakes', 'Hamster Bedding']
['Dog Treats', 'Cat Litter', 'Dog Collar', 'Cat Toy', 'Aquarium Kit']
['Bird Cage', 'Hamster Cage', 'Pet Shampoo', 'Leash', 'Pet Bed']
['Fleece Blanket', 'Water Bottle', 'Fish Tank Decor', 'Nail Clippers']
['Grooming Brush', 'Puppy Pads', 'Reptile Habitat', 'Small Animal Carrier']
['Treat Dispenser', 'Chew Toys', 'Scratch Post', 'Automatic Feeder', 'Litter Box']
['Dog Training Pads', 'Dog Food', 'Cat Food', 'Bird Seed']
['Cat Litter', 'Dog Collar', 'Dog Toy', 'Aquarium Kit', 'Bird Cage']
['Hamster Cage', 'Pet Shampoo', 'Leash', 'Pet Bed', 'Fleece Blanket']
['Water Bottle', 'Fish Tank Decor', 'Nail Clippers', 'Grooming Brush']
['Puppy Pads', 'Reptile Habitat', 'Small Animal Carrier', 'Treat Dispenser']
['Chew Toys', 'Scratch Post', 'Automatic Feeder', 'Litter Box', 'Dog Training Pads']
['Dog Food', 'Cat Food', 'Bird Seed', 'Fish Flakes', 'Hamster Bedding', 'Dog Treats']
['Cat Litter', 'Dog Collar', 'Cat Toy', 'Dog Toy', 'Aquarium Kit']
['Bird Cage', 'Hamster Cage', 'Pet Shampoo', 'Leash', 'Pet Bed']
['Fleece Blanket', 'Water Bottle', 'Fish Tank Decor', 'Nail Clippers']
['Grooming Brush', 'Puppy Pads', 'Reptile Habitat', 'Small Animal Carrier']
['Treat Dispenser', 'Chew Toys', 'Scratch Post', 'Automatic Feeder', 'Litter Box']
['Dog Training Pads', 'Dog Food', 'Cat Food', 'Bird Seed']
['Dog Training Pads', 'Dog Food', 'Cat Food', 'Small Animal Carrier']

```

```

Apriori Frequent Itemsets
support      itemsets  length
0 0.20      (Bird Seed) 1
1 0.25      (Cat Food) 1
2 0.25      (Dog Food) 1
3 0.20      (Dog Training Pads) 1
4 0.20      (Small Animal Carrier) 1
5 0.20      (Cat Food, Bird Seed) 2
6 0.20      (Bird Seed, Dog Food) 2
7 0.25      (Cat Food, Dog Food) 2
8 0.20      (Cat Food, Bird Seed, Dog Food) 3

```

```

Apriori Rules
antecedents      consequents  antecedent support  consequent support  support  confidence  lift  leverage  conviction  zhangs_metric
0      (Cat Food)      (Bird Seed) 0.25      0.20      0.20      0.8      4.0      0.1500      4.0      1.0000
1      (Bird Seed)      (Cat Food) 0.20      0.25      0.20      1.0      4.0      0.1500      inf      0.9375
2      (Bird Seed)      (Dog Food) 0.20      0.25      0.20      1.0      4.0      0.1500      inf      0.9375
3      (Dog Food)      (Bird Seed) 0.25      0.20      0.20      0.8      4.0      0.1500      4.0      1.0000
4      (Cat Food)      (Dog Food) 0.25      0.25      0.25      1.0      4.0      0.1875      inf      1.0000
5      (Dog Food)      (Cat Food) 0.25      0.25      0.25      1.0      4.0      0.1875      inf      1.0000
6      (Cat Food, Bird Seed)      (Dog Food) 0.20      0.25      0.20      1.0      4.0      0.1500      inf      0.9375
7      (Cat Food, Dog Food)      (Bird Seed) 0.25      0.20      0.20      0.8      4.0      0.1500      4.0      1.0000
8      (Dog Food, Bird Seed)      (Cat Food) 0.20      0.25      0.20      1.0      4.0      0.1500      inf      0.9375
9      (Cat Food)      (Dog Food, Bird Seed) 0.25      0.20      0.20      0.8      4.0      0.1500      4.0      1.0000
10     (Bird Seed)      (Cat Food, Dog Food) 0.20      0.25      0.20      1.0      4.0      0.1500      inf      0.9375
11     (Dog Food)      (Cat Food, Bird Seed) 0.25      0.20      0.20      0.8      4.0      0.1500      4.0      1.0000

```

```

Brute Force Frequent Itemsets
combo      support  length
0      (Bird Seed,) 0.20      1
1      (Cat Food,) 0.25      1
2      (Dog Food,) 0.25      1
3      (Dog Training Pads,) 0.20      1
4      (Small Animal Carrier,) 0.20      1
5      (Bird Seed, Cat Food) 0.20      2
6      (Bird Seed, Dog Food) 0.20      2
7      (Cat Food, Dog Food) 0.25      2
8      (Bird Seed, Cat Food, Dog Food) 0.20      3

```

```

Brute Force Rules
antecedents      consequents  antecedent support  consequent support  support  confidence
0      (Bird Seed,)      (Cat Food,) 0.20      0.25      0.20      1.0
1      (Cat Food,)      (Bird Seed,) 0.25      0.20      0.20      0.8
2      (Bird Seed,)      (Dog Food,) 0.20      0.25      0.20      1.0
3      (Dog Food,)      (Bird Seed,) 0.25      0.20      0.20      0.8
4      (Cat Food,)      (Dog Food,) 0.25      0.25      0.25      1.0
5      (Dog Food,)      (Cat Food,) 0.25      0.25      0.25      1.0
6      (Bird Seed,)      (Cat Food, Dog Food) 0.20      0.25      0.20      1.0
11     (Cat Food,)      (Bird Seed, Dog Food) 0.25      0.20      0.20      0.8
14     (Dog Food,)      (Bird Seed, Cat Food) 0.25      0.20      0.20      0.8
15     (Bird Seed, Cat Food)      (Dog Food,) 0.20      0.25      0.20      1.0
16     (Bird Seed, Dog Food)      (Cat Food,) 0.20      0.25      0.20      1.0
17     (Cat Food, Dog Food)      (Bird Seed,) 0.25      0.20      0.20      0.8

```

```

Apriori Time: 0.015625
Brute Force Time: 10.828125
Apriori took 0.1443001443001443 percent as long as Brute Force

```

projdata4

```

Transactions:
['Chocolate Bar', 'Gummy Bears', 'Hard Candy', 'Lollipops', 'Sour Candy']
['Licorice', 'Jelly Beans', 'Candy Corn', 'Mints', 'Taffy']
['Caramel', 'Marshmallows', 'Cotton Candy', 'Fruit Chews', 'Chocolate Covered Nuts']
['Toffee', 'Chocolate-Covered Pretzels', 'Fudge', 'Peanut Butter Cups', 'Rice Crispy Treats']
['Bonbons', 'Candy Necklace', 'Spicy Candy', 'Bubblegum']
['Chocolate-Covered Fruit', 'Sugar-Free Candy', 'Chocolate Truffles', 'Whirly Pop', 'Almond Joy']
['Swedish Fish', 'Chocolate Bar', 'Gummy Bears', 'Hard Candy', 'Lollipops']
['Sour Candy', 'Licorice', 'Jelly Beans', 'Candy Corn', 'Mints']
['Taffy', 'Caramel', 'Marshmallows', 'Cotton Candy', 'Fruit Chews']
['Chocolate Covered Nuts', 'Toffee', 'Chocolate-Covered Pretzels', 'Fudge', 'Peanut Butter Cups']
['Rice Crispy Treats', 'Bonbons', 'Candy Necklace', 'Spicy Candy']
['Bubblegum', 'Chocolate-Covered Fruit', 'Sugar-Free Candy', 'Chocolate Truffles', 'Whirly Pop']
['Almond Joy', 'Swedish Fish', 'Chocolate Bar', 'Gummy Bears', 'Hard Candy']
['Lollipops', 'Sour Candy', 'Licorice', 'Jelly Beans', 'Candy Corn']
['Mints', 'Taffy', 'Caramel', 'Marshmallows', 'Cotton Candy']
['Fruit Chews', 'Chocolate Covered Nuts', 'Toffee', 'Chocolate-Covered Pretzels']
['Peanut Butter Cups', 'Rice Crispy Treats', 'Bonbons', 'Candy Necklace']
['Chocolate Bar', 'Gummy Bears', 'Hard Candy', 'Lollipops', 'Sour Candy']
['Licorice', 'Jelly Beans', 'Candy Corn', 'Mints', 'Taffy']
['Caramel', 'Marshmallows', 'Cotton Candy', 'Fruit Chews', 'Chocolate Covered Nuts']

```

```

Apriori Frequent Itemsets
support      itemsets      length
0 0.2 (Candy Corn) 1
1 0.2 (Caramel) 1
2 0.2 (Chocolate Bar) 1
3 0.2 (Chocolate Covered Nuts) 1
4 0.2 (Cotton Candy) 1
5 0.2 (Fruit Chews) 1
6 0.2 (Gummy Bears) 1
7 0.2 (Hard Candy) 1
8 0.2 (Jelly Beans) 1
9 0.2 (Licorice) 1
10 0.2 (Lollipops) 1
11 0.2 (Marshmallows) 1
12 0.2 (Mints) 1
13 0.2 (Sour Candy) 1
14 0.2 (Taffy) 1
15 0.2 (Candy Corn, Jelly Beans) 2
16 0.2 (Licorice, Candy Corn) 2
17 0.2 (Caramel, Cotton Candy) 2
18 0.2 (Marshmallows, Caramel) 2
19 0.2 (Gummy Bears, Chocolate Bar) 2
20 0.2 (Hard Candy, Chocolate Bar) 2
21 0.2 (Marshmallows, Cotton Candy) 2
22 0.2 (Gummy Bears, Hard Candy) 2
23 0.2 (Licorice, Jelly Beans) 2
24 0.2 (Licorice, Candy Corn, Jelly Beans) 3
25 0.2 (Marshmallows, Caramel, Cotton Candy) 3
26 0.2 (Hard Candy, Chocolate Bar, Gummy Bears) 3

Apriori Rules
antecedents      consequents      antecedent support      consequent support      support      confidence      lift      leverage      conviction      zhangs_metric
0 (Candy Corn) (Jelly Beans) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
1 (Jelly Beans) (Candy Corn) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
2 (Licorice) (Candy Corn) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
3 (Candy Corn) (Licorice) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
4 (Caramel) (Cotton Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
5 (Cotton Candy) (Caramel) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
6 (Marshmallows) (Caramel) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
7 (Caramel) (Marshmallows) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
8 (Gummy Bears) (Chocolate Bar) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
9 (Chocolate Bar) (Gummy Bears) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
10 (Hard Candy) (Chocolate Bar) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
11 (Chocolate Bar) (Hard Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
12 (Marshmallows) (Cotton Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
13 (Cotton Candy) (Marshmallows) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
14 (Gummy Bears) (Hard Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
15 (Hard Candy) (Gummy Bears) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
16 (Licorice) (Jelly Beans) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
17 (Jelly Beans) (Licorice) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
18 (Licorice, Candy Corn) (Jelly Beans) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
19 (Licorice, Jelly Beans) (Candy Corn) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
20 (Candy Corn, Jelly Beans) (Licorice) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
21 (Licorice) (Candy Corn, Jelly Beans) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
22 (Candy Corn) (Licorice, Jelly Beans) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
23 (Jelly Beans) (Licorice, Candy Corn) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
24 (Marshmallows, Caramel) (Cotton Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
25 (Marshmallows, Cotton Candy) (Caramel) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
26 (Caramel, Cotton Candy) (Marshmallows) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
27 (Marshmallows) (Caramel, Cotton Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
28 (Caramel) (Marshmallows, Cotton Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
29 (Cotton Candy) (Marshmallows, Caramel) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
30 (Hard Candy, Chocolate Bar) (Gummy Bears) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
31 (Hard Candy, Gummy Bears) (Chocolate Bar) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
32 (Gummy Bears, Chocolate Bar) (Hard Candy) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
33 (Hard Candy) (Gummy Bears, Chocolate Bar) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
34 (Chocolate Bar) (Hard Candy, Gummy Bears) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0
35 (Gummy Bears) (Hard Candy, Chocolate Bar) 0.2 0.2 0.2 1.0 5.0 0.16 inf 1.0

```

Brute Force Frequent Itemsets			
	combo	support	length
0	(Candy Corn,)	0.2	1
1	(Caramel,)	0.2	1
2	(Chocolate Bar,)	0.2	1
3	(Chocolate Covered Nuts,)	0.2	1
4	(Cotton Candy,)	0.2	1
5	(Fruit Chews,)	0.2	1
6	(Gummy Bears,)	0.2	1
7	(Hard Candy,)	0.2	1
8	(Jelly Beans,)	0.2	1
9	(Licorice,)	0.2	1
10	(Lollipops,)	0.2	1
11	(Marshmallows,)	0.2	1
12	(Mints,)	0.2	1
13	(Sour Candy,)	0.2	1
14	(Taffy,)	0.2	1
15	(Candy Corn, Jelly Beans)	0.2	2
16	(Candy Corn, Licorice)	0.2	2
17	(Caramel, Cotton Candy)	0.2	2
18	(Caramel, Marshmallows)	0.2	2
19	(Chocolate Bar, Gummy Bears)	0.2	2
20	(Chocolate Bar, Hard Candy)	0.2	2
21	(Cotton Candy, Marshmallows)	0.2	2
22	(Gummy Bears, Hard Candy)	0.2	2
23	(Jelly Beans, Licorice)	0.2	2
24	(Candy Corn, Jelly Beans, Licorice)	0.2	3
25	(Caramel, Cotton Candy, Marshmallows)	0.2	3
26	(Chocolate Bar, Gummy Bears, Hard Candy)	0.2	3

Brute Force Rules						
	antecedents	consequents	antecedent support	consequent support	support	confidence
0	(Candy Corn,)	(Jelly Beans,)	0.2	0.2	0.2	1.0
1	(Jelly Beans,)	(Candy Corn,)	0.2	0.2	0.2	1.0
2	(Candy Corn,)	(Licorice,)	0.2	0.2	0.2	1.0
3	(Licorice,)	(Candy Corn,)	0.2	0.2	0.2	1.0
4	(Caramel,)	(Cotton Candy,)	0.2	0.2	0.2	1.0
5	(Cotton Candy,)	(Caramel,)	0.2	0.2	0.2	1.0
6	(Caramel,)	(Marshmallows,)	0.2	0.2	0.2	1.0
7	(Marshmallows,)	(Caramel,)	0.2	0.2	0.2	1.0
8	(Chocolate Bar,)	(Gummy Bears,)	0.2	0.2	0.2	1.0
9	(Gummy Bears,)	(Chocolate Bar,)	0.2	0.2	0.2	1.0
10	(Chocolate Bar,)	(Hard Candy,)	0.2	0.2	0.2	1.0
11	(Hard Candy,)	(Chocolate Bar,)	0.2	0.2	0.2	1.0
12	(Cotton Candy,)	(Marshmallows,)	0.2	0.2	0.2	1.0
13	(Marshmallows,)	(Cotton Candy,)	0.2	0.2	0.2	1.0
14	(Gummy Bears,)	(Hard Candy,)	0.2	0.2	0.2	1.0
15	(Hard Candy,)	(Gummy Bears,)	0.2	0.2	0.2	1.0
16	(Jelly Beans,)	(Licorice,)	0.2	0.2	0.2	1.0
17	(Licorice,)	(Jelly Beans,)	0.2	0.2	0.2	1.0
20	(Candy Corn,)	(Jelly Beans, Licorice)	0.2	0.2	0.2	1.0
23	(Jelly Beans,)	(Candy Corn, Licorice)	0.2	0.2	0.2	1.0
26	(Licorice,)	(Candy Corn, Jelly Beans)	0.2	0.2	0.2	1.0
27	(Candy Corn, Jelly Beans)	(Licorice,)	0.2	0.2	0.2	1.0
28	(Candy Corn, Licorice)	(Jelly Beans,)	0.2	0.2	0.2	1.0
29	(Jelly Beans, Licorice)	(Candy Corn,)	0.2	0.2	0.2	1.0
32	(Caramel,)	(Cotton Candy, Marshmallows)	0.2	0.2	0.2	1.0
35	(Cotton Candy,)	(Caramel, Marshmallows)	0.2	0.2	0.2	1.0
38	(Marshmallows,)	(Caramel, Cotton Candy)	0.2	0.2	0.2	1.0
39	(Caramel, Cotton Candy)	(Marshmallows,)	0.2	0.2	0.2	1.0
40	(Caramel, Marshmallows)	(Cotton Candy,)	0.2	0.2	0.2	1.0
41	(Cotton Candy, Marshmallows)	(Caramel,)	0.2	0.2	0.2	1.0
44	(Chocolate Bar,)	(Gummy Bears, Hard Candy)	0.2	0.2	0.2	1.0
47	(Gummy Bears,)	(Chocolate Bar, Hard Candy)	0.2	0.2	0.2	1.0
50	(Hard Candy,)	(Chocolate Bar, Gummy Bears)	0.2	0.2	0.2	1.0
51	(Chocolate Bar, Gummy Bears)	(Hard Candy,)	0.2	0.2	0.2	1.0
52	(Chocolate Bar, Hard Candy)	(Gummy Bears,)	0.2	0.2	0.2	1.0
53	(Gummy Bears, Hard Candy)	(Chocolate Bar,)	0.2	0.2	0.2	1.0

Apriori Time: 0.0
 Brute Force Time: 11.0
 Apriori took 0.0 percent as long as Brute Force

projdata5

```

Transactions:
['Pain Reliever', 'Cough Syrup', 'Allergy Medication', 'Vitamins', 'Cold Medicine']
['Antacid', 'First Aid Kit', 'Band-Aids', 'Thermometer', 'Hand Sanitizer']
['Prescription Medication', 'Sunscreen', 'Moisturizer', 'Lip Balm', 'Hydrocortisone Cream']
['Eye Drops', 'Saline Nasal Spray', 'Digestive Enzyme', 'Anti-Itch Cream', 'Diabetic Test Strips']
['Insect Bite Relief', 'Oral Rehydration Salts', 'Sleeping Aid', 'Antiseptic Wipes', 'Muscle Rub']
['Wart Remover', 'Hair Loss Treatment', 'Decongestant']
['Skin Ointment', 'Pain Reliever', 'Cough Syrup', 'Allergy Medication', 'Vitamins']
['Cold Medicine', 'Antacid', 'First Aid Kit', 'Band-Aids', 'Thermometer']
['Hand Sanitizer', 'Prescription Medication', 'Sunscreen', 'Moisturizer', 'Lip Balm']
['Hydrocortisone Cream', 'Eye Drops', 'Saline Nasal Spray', 'Digestive Enzyme', 'Anti-Itch Cream']
['Diabetic Test Strips', 'Insect Bite Relief', 'Oral Rehydration Salts', 'Sleeping Aid', 'Antiseptic Wipes']
['Muscle Rub', 'Wart Remover', 'Hair Loss Treatment']
['Decongestant', 'Pain Reliever', 'Cough Syrup', 'Allergy Medication', 'Vitamins']
['Cold Medicine', 'Antacid', 'First Aid Kit', 'Band-Aids', 'Thermometer']
['Hand Sanitizer', 'Prescription Medication', 'Sunscreen', 'Moisturizer', 'Lip Balm']
['Hydrocortisone Cream', 'Eye Drops', 'Saline Nasal Spray', 'Digestive Enzyme', 'Anti-Itch Cream']
['Diabetic Test Strips', 'Insect Bite Relief', 'Oral Rehydration Salts', 'Sleeping Aid', 'Antiseptic Wipes']
['Muscle Rub', 'Wart Remover', 'Hair Loss Treatment']
['Decongestant', 'Pain Reliever', 'Cough Syrup', 'Allergy Medication']
['Cold Medicine', 'Antacid', 'First Aid Kit', 'Band-Aids', 'Thermometer']

```

Apriori Frequent Itemsets

	support	itemsets	length
0	0.2	(Allergy Medication)	1
1	0.2	(Antacid)	1
2	0.2	(Band-Aids)	1
3	0.2	(Cold Medicine)	1
4	0.2	(Cough Syrup)	1
5	0.2	(First Aid Kit)	1
6	0.2	(Pain Reliever)	1
7	0.2	(Thermometer)	1
8	0.2	(Allergy Medication, Cough Syrup)	2
9	0.2	(Allergy Medication, Pain Reliever)	2
10	0.2	(Antacid, Band-Aids)	2
11	0.2	(Antacid, First Aid Kit)	2
12	0.2	(Antacid, Thermometer)	2
13	0.2	(Band-Aids, First Aid Kit)	2
14	0.2	(Thermometer, Band-Aids)	2
15	0.2	(Cough Syrup, Pain Reliever)	2
16	0.2	(Thermometer, First Aid Kit)	2
17	0.2	(Allergy Medication, Cough Syrup, Pain Reliever)	3
18	0.2	(Antacid, Band-Aids, First Aid Kit)	3
19	0.2	(Antacid, Thermometer, Band-Aids)	3
20	0.2	(Thermometer, Antacid, First Aid Kit)	3
21	0.2	(Thermometer, Band-Aids, First Aid Kit)	3
22	0.2	(Thermometer, Band-Aids, Antacid, First Aid Kit)	4

Apriori Rules											
	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction	zhangs_metric	
0	(Allergy Medication)	(Cough Syrup)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
1	(Cough Syrup)	(Allergy Medication)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
2	(Allergy Medication)	(Pain Reliever)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
3	(Pain Reliever)	(Allergy Medication)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
4	(Antacid)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
5	(Band-Aids)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
6	(Antacid)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
7	(First Aid Kit)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
8	(Antacid)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
9	(Thermometer)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
10	(Band-Aids)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
11	(First Aid Kit)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
12	(Thermometer)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
13	(Band-Aids)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
14	(Cough Syrup)	(Pain Reliever)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
15	(Pain Reliever)	(Cough Syrup)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
16	(Thermometer)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
17	(First Aid Kit)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
18	(Allergy Medication, Cough Syrup)	(Pain Reliever)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
19	(Allergy Medication, Pain Reliever)	(Cough Syrup)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
20	(Pain Reliever, Cough Syrup)	(Allergy Medication)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
21	(Allergy Medication)	(Pain Reliever, Cough Syrup)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
22	(Cough Syrup)	(Allergy Medication, Pain Reliever)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
23	(Pain Reliever)	(Allergy Medication, Cough Syrup)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
24	(Antacid, Band-Aids)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
25	(Antacid, First Aid Kit)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
26	(Band-Aids, First Aid Kit)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
27	(Antacid)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
28	(Band-Aids)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
29	(First Aid Kit)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
30	(Antacid, Thermometer)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
31	(Antacid, Band-Aids)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
32	(Thermometer, Band-Aids)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
33	(Antacid)	(Thermometer, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
34	(Thermometer)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
35	(Band-Aids)	(Antacid, Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
36	(Thermometer, Antacid)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
37	(Thermometer, First Aid Kit)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
38	(Antacid, First Aid Kit)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
39	(Thermometer)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
40	(Antacid)	(Thermometer, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
41	(First Aid Kit)	(Thermometer, Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
42	(Thermometer, Band-Aids)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
43	(Thermometer, First Aid Kit)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
44	(Band-Aids, First Aid Kit)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
45	(Thermometer)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
46	(Band-Aids)	(Thermometer, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
47	(First Aid Kit)	(Thermometer, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
48	(Thermometer, Band-Aids, Antacid)	(First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
49	(Thermometer, Band-Aids, First Aid Kit)	(Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
50	(Thermometer, Antacid, First Aid Kit)	(Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
51	(Antacid, Band-Aids, First Aid Kit)	(Thermometer)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
52	(Thermometer, Band-Aids)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
53	(Thermometer, Antacid)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
54	(Thermometer, First Aid Kit)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
55	(Antacid, Band-Aids)	(Thermometer, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
56	(Band-Aids, First Aid Kit)	(Thermometer, Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
57	(Antacid, First Aid Kit)	(Thermometer, Band-Aids)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
58	(Thermometer)	(Antacid, Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
59	(Band-Aids)	(Thermometer, Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
60	(Antacid)	(Thermometer, Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	
61	(First Aid Kit)	(Thermometer, Band-Aids, Antacid)	0.2	0.2	0.2	1.0	5.0	0.16	inf	1.0	

Brute Force Frequent Itemsets

	combo	support	length
0	(Allergy Medication,)	0.2	1
1	(Antacid,)	0.2	1
2	(Band-Aids,)	0.2	1
3	(Cold Medicine,)	0.2	1
4	(Cough Syrup,)	0.2	1
5	(First Aid Kit,)	0.2	1
6	(Pain Reliever,)	0.2	1
7	(Thermometer,)	0.2	1
8	(Allergy Medication, Cough Syrup)	0.2	2
9	(Allergy Medication, Pain Reliever)	0.2	2
10	(Antacid, Band-Aids)	0.2	2
11	(Antacid, First Aid Kit)	0.2	2
12	(Antacid, Thermometer)	0.2	2
13	(Band-Aids, First Aid Kit)	0.2	2
14	(Band-Aids, Thermometer)	0.2	2
15	(Cough Syrup, Pain Reliever)	0.2	2
16	(First Aid Kit, Thermometer)	0.2	2
17	(Allergy Medication, Cough Syrup, Pain Reliever)	0.2	3
18	(Antacid, Band-Aids, First Aid Kit)	0.2	3
19	(Antacid, Band-Aids, Thermometer)	0.2	3
20	(Antacid, First Aid Kit, Thermometer)	0.2	3
21	(Band-Aids, First Aid Kit, Thermometer)	0.2	3
22	(Antacid, Band-Aids, First Aid Kit, Thermometer)	0.2	4

Brute Force Rules							
	antecedents	consequents	antecedent support	consequent support	support	confidence	
0	(Allergy Medication,)	(Cough Syrup,)	0.2	0.2	0.2	1.0	
1	(Cough Syrup,)	(Allergy Medication,)	0.2	0.2	0.2	1.0	
2	(Allergy Medication,)	(Pain Reliever,)	0.2	0.2	0.2	1.0	
3	(Pain Reliever,)	(Allergy Medication,)	0.2	0.2	0.2	1.0	
4	(Antacid,)	(Band-Aids,)	0.2	0.2	0.2	1.0	
5	(Band-Aids,)	(Antacid,)	0.2	0.2	0.2	1.0	
6	(Antacid,)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
7	(First Aid Kit,)	(Antacid,)	0.2	0.2	0.2	1.0	
8	(Antacid,)	(Thermometer,)	0.2	0.2	0.2	1.0	
9	(Thermometer,)	(Antacid,)	0.2	0.2	0.2	1.0	
10	(Band-Aids,)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
11	(First Aid Kit,)	(Band-Aids,)	0.2	0.2	0.2	1.0	
12	(Band-Aids,)	(Thermometer,)	0.2	0.2	0.2	1.0	
13	(Thermometer,)	(Band-Aids,)	0.2	0.2	0.2	1.0	
14	(Cough Syrup,)	(Pain Reliever,)	0.2	0.2	0.2	1.0	
15	(Pain Reliever,)	(Cough Syrup,)	0.2	0.2	0.2	1.0	
16	(First Aid Kit,)	(Thermometer,)	0.2	0.2	0.2	1.0	
17	(Thermometer,)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
20	(Allergy Medication,)	(Cough Syrup, Pain Reliever)	0.2	0.2	0.2	1.0	
23	(Cough Syrup,)	(Allergy Medication, Pain Reliever)	0.2	0.2	0.2	1.0	
26	(Pain Reliever,)	(Allergy Medication, Cough Syrup)	0.2	0.2	0.2	1.0	
27	(Allergy Medication, Cough Syrup)	(Pain Reliever,)	0.2	0.2	0.2	1.0	
28	(Allergy Medication, Pain Reliever)	(Cough Syrup,)	0.2	0.2	0.2	1.0	
29	(Cough Syrup, Pain Reliever)	(Allergy Medication,)	0.2	0.2	0.2	1.0	
32	(Antacid,)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	
35	(Band-Aids,)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	
38	(First Aid Kit,)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	
39	(Antacid, Band-Aids)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
40	(Antacid, First Aid Kit)	(Band-Aids,)	0.2	0.2	0.2	1.0	
41	(Band-Aids, First Aid Kit)	(Antacid,)	0.2	0.2	0.2	1.0	
44	(Antacid,)	(Band-Aids, Thermometer)	0.2	0.2	0.2	1.0	
47	(Band-Aids,)	(Antacid, Thermometer)	0.2	0.2	0.2	1.0	
50	(Thermometer,)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	
51	(Antacid, Band-Aids)	(Thermometer,)	0.2	0.2	0.2	1.0	
52	(Antacid, Thermometer)	(Band-Aids,)	0.2	0.2	0.2	1.0	
53	(Band-Aids, Thermometer)	(Antacid,)	0.2	0.2	0.2	1.0	
56	(Antacid,)	(First Aid Kit, Thermometer)	0.2	0.2	0.2	1.0	
59	(First Aid Kit,)	(Antacid, Thermometer)	0.2	0.2	0.2	1.0	
62	(Thermometer,)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	
63	(Antacid, First Aid Kit)	(Thermometer,)	0.2	0.2	0.2	1.0	
64	(Antacid, Thermometer)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
65	(First Aid Kit, Thermometer)	(Antacid,)	0.2	0.2	0.2	1.0	
68	(Band-Aids,)	(First Aid Kit, Thermometer)	0.2	0.2	0.2	1.0	
71	(First Aid Kit,)	(Band-Aids, Thermometer)	0.2	0.2	0.2	1.0	
74	(Thermometer,)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	
75	(Band-Aids, First Aid Kit)	(Thermometer,)	0.2	0.2	0.2	1.0	
76	(Band-Aids, Thermometer)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
77	(First Aid Kit, Thermometer)	(Band-Aids,)	0.2	0.2	0.2	1.0	
84	(Antacid,)	(Band-Aids, First Aid Kit, Thermometer)	0.2	0.2	0.2	1.0	
91	(Band-Aids,)	(Antacid, First Aid Kit, Thermometer)	0.2	0.2	0.2	1.0	
98	(First Aid Kit,)	(Antacid, Band-Aids, Thermometer)	0.2	0.2	0.2	1.0	
105	(Thermometer,)	(Antacid, Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	
108	(Antacid, Band-Aids)	(First Aid Kit, Thermometer)	0.2	0.2	0.2	1.0	
111	(Antacid, First Aid Kit)	(Band-Aids, Thermometer)	0.2	0.2	0.2	1.0	
114	(Antacid, Thermometer)	(Band-Aids, First Aid Kit)	0.2	0.2	0.2	1.0	
117	(Band-Aids, First Aid Kit)	(Antacid, Thermometer)	0.2	0.2	0.2	1.0	
120	(Band-Aids, Thermometer)	(Antacid, First Aid Kit)	0.2	0.2	0.2	1.0	
123	(First Aid Kit, Thermometer)	(Antacid, Band-Aids)	0.2	0.2	0.2	1.0	
124	(Antacid, Band-Aids, First Aid Kit)	(Thermometer,)	0.2	0.2	0.2	1.0	
125	(Antacid, Band-Aids, Thermometer)	(First Aid Kit,)	0.2	0.2	0.2	1.0	
126	(Antacid, First Aid Kit, Thermometer)	(Band-Aids,)	0.2	0.2	0.2	1.0	
127	(Band-Aids, First Aid Kit, Thermometer)	(Antacid,)	0.2	0.2	0.2	1.0	

Apriori Time: 0.015625

Brute Force Time: 67.640625

Apriori took 0.0231000231000231 percent as long as Brute Force